

Yolov8 模型训练与部署



版本历史

版本	版本更新说明	责任人	校审人	发布时间
V1.0	初次	易百纳板卡团队	易百纳板卡团队	2025 年 10 月 28 日

易百纳技术社区 Ebaina 易百纳技术社区 Ebaina 易百纳技术社区 Ebaina 易百纳技术社区 Ebaina

免责声明

本文档提供有关南京启诺信息技术有限公司产品的信息，未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。本文档所陈述的产品文本及相关软件版权 均属南京启诺信息技术有限公司所有，其产权受国家法律绝对保护，未经本公司授权，其它公司、单位、代理商及个人不得非法使用和拷贝，否则将受到国家法律的严厉制裁。

为了得到最新版本的产品信息，请用户定时访问易百纳技术社区网站平台下载或者与淘宝易百纳技术社区客服联系索取。感谢您的包容与支持。

目录

1. YOLOv8 环境搭建	4
1.1. 创建虚拟环境	4
1.1.1. 安装 Miniconda	4
1.1.2. conda 命令创建虚拟环境	4
1.2. 环境搭建	5
1.2.1. 安装依赖、补丁	5
1.2.2. 对应 python 和 cuda 版本	6
1.2.3. 环境测试	6
2. 训练数据集制作	6
2.1. 数据集格式介绍	7
2.2. 数据集标注	8
3. YOLOv8 训练	8
3.1. 查看显卡	8
3.2. 上传数据集到服务器上	8
3.3. 修改 yaml 文件	9
3.4. 训练	10
3.5. 预测	10
3.6. onnx 模型导出	10
4. onnx 模型转换	11
4.1. 安装 atc 相关环境	11
4.2. 安装 atc 相关依赖	11
4.3. 安装 atc 软件	11
4.4. YOLOV8 转换模型	12
4.5. op.cfg 文件	12
5. 板端推理	13
5.1. 运行方式	13

1. YOLOv8 环境搭建

1.1. 创建虚拟环境

通过 conda，为 YOLOV8 安装创建一个干净的环境。

1.1.1. 安装 Miniconda

conda 允许用户设置独立的环境来运行不同版本的 Python 或其他语言，而无需切换到其他环境管理器。Miniconda 和 Anaconda 均是 conda 工具，但 Miniconda 是 Anaconda 最小安装程序，内存占用较小，YOLOv8 的训练环境搭建将使用 conda 工具创建一个独立的 python 环境。

执行下面命令安装 Miniconda3

```
#更新 apt 源
apt-get update
#下载 miniconda
wget https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-latest
Linux-x86_64.sh
#添加可执行权限
chmod +x Miniconda3-latest-Linux-x86_64.sh
#执行安装
./Miniconda3-latest-Linux-x86_64.sh -b -p ~/miniconda3
```

1.1.2. conda 命令创建虚拟环境

下面命令为 conda 环境的基本命令，请根据前 5 条命令创建并切换到虚拟环境内。

```
#初始化 miniconda
cd ~/miniconda3/bin/
./conda init
echo 'conda deactivate' >> ~/.bashrc
#重载环境
source ~/.bashrc
#列出所有环境
conda env list
#创建虚拟环境,[env_name]为环境名，这里选择 python 版本为 3.10，可选择其他版本，但需满足
```

```
python>=3.8
conda create -n [env_name] python=3.10
#切换环境
conda activate [env_name]
#退出环境
conda deactivate
#删除环境
conda env remove -n [env_name]
```

1. 2. 环境搭建

1. 2. 1. 安装依赖、补丁

下载 yolov8 代码。

```
mkdir yolov8 && cd yolov8
git clone https://github.com/ultralytics/ultralytics.git
```

在源码路径中运行命令，加载指定版本。

```
git reset --hard d3f097314f9478de7f995d4e4b4ccb0c6fbc65d3
cd ultralytics
```

下载模型至源码目录下，下载路径。

wget <https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8n.pt>

在 yolov8 源码路径中创建 requirements_yolov8.txt 文件，添加以下内容。

```
# Usage: pip install -r requirements_yolov8.txt
# Base -----
matplotlib>=3.2.2
numpy>=1.21.6
opencv-python>=4.6.0
Pillow>=7.1.2
PyYAML>=5.3.1
requests>=2.23.0
scipy>=1.4.1
torch==1.12.0
torchvision==0.13.0
tqdm>=4.64.0
# Plotting -----
pandas>=1.1.4
```

```
seaborn>=0.11.0
# Export -----
onnx>=1.12.0 # ONNX export
onnxsim>=0.4.1 # ONNX simplifier
# Extras -----
psutil # system utilization
thop>=0.1.1 # FLOPs computation
#-----
```

安装依赖，在 yolov8 源码 ultralytics 目录下执行

```
pip install --upgrade pip
pip install -r requirements_yolov8.txt
```

安装 ultralytics，ultralytics 版本必须为 8.0.68

```
pip install ultralytics==8.0.68
```

1.2.2. 对应 python 和 cuda 版本

查看 cuda 版本。

```
nvcc -V
```

如果出现 nvcc no found，在 ~/.bashrc 文件末尾添加如下环境变量。

```
export PATH="/usr/local/cuda/bin:$PATH"
```

1.2.3. 环境测试

在服务器 yolov8 源码目录下创建一个 test.py 文件，添加以下内容：

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
success = model.export(format="onnx", opset=12)
```

执行 test.py 文件导出原始 onnx 模型，若无报错则代表当前环境正常。

```
python test.py
```

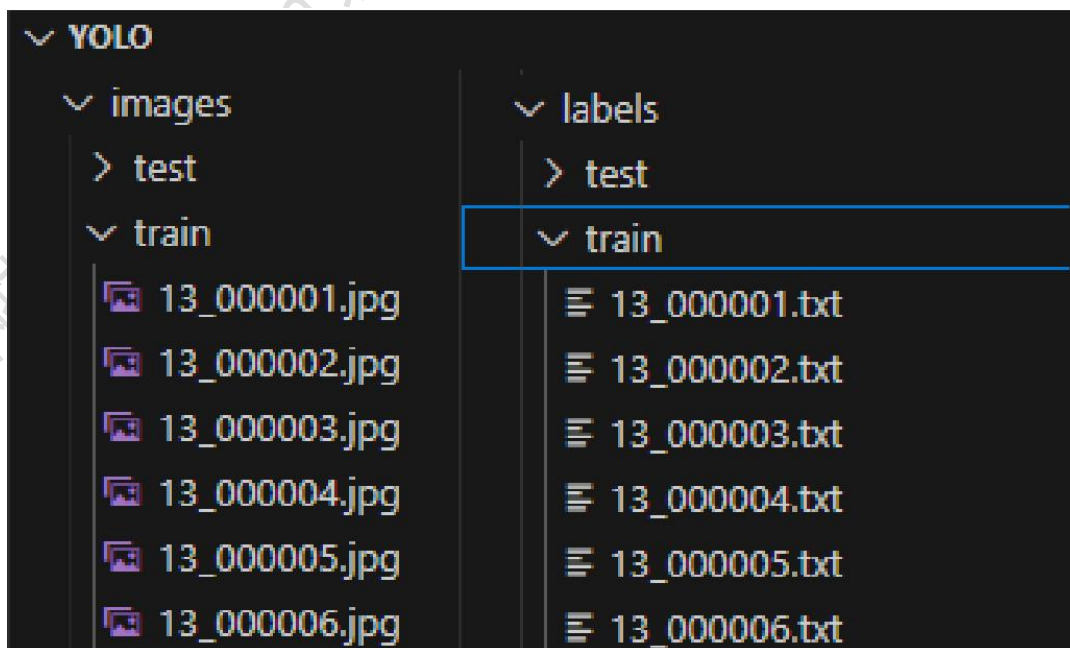
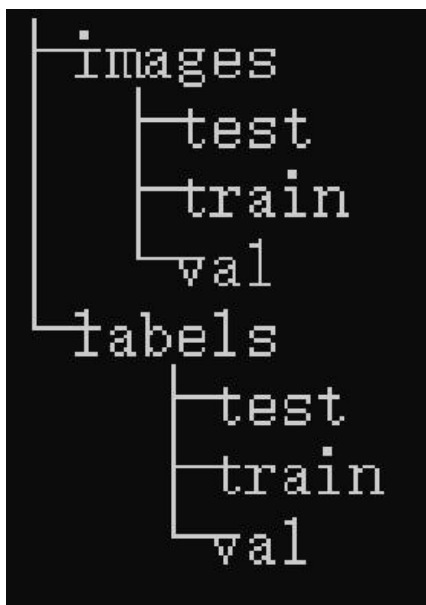
2. 训练数据集制作

数据集用于模型训练。

2.1. 数据集格式介绍

yolov8 的数据集格式是 yolo(txt)格式，整个数据集文件夹先是分为图片和标签文件夹，分别存储图片和标签文件，而图片和标签文件夹下均再继续分为训练集、验证集和测试集，训练时训练集和验证集是必须的，测试集后续预测才需要。

数据集文件目录格式如下，images 存放图片，labels 存放标签文件，其中两者的子目录：train 为训练集，val 为验证集，test 为测试集；两者的子目录下存放的图片和标签文件除扩展名外名称是一样，是相互对应的。

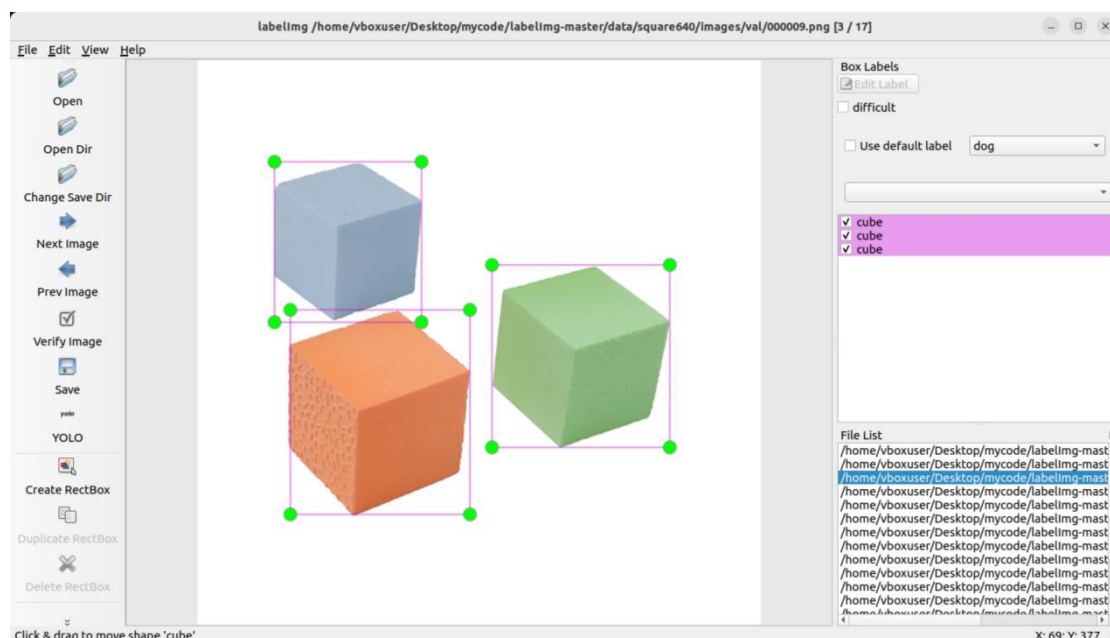


2.2. 数据集标注

yolov8 数据集采用 yolo(txt)格式，采用 labellmg 工具标注，选择输出 yolo 格式。

labellmg 安装方法见：<https://cloud.tencent.com/developer/article/2039112>

数据标注方法如下图：



首先打开图片目录，选择输出格式为 YOLO，点击 create RectBox 按钮为图形做标注。

3. YOLOv8 训练

请先登录服务器并进入 docke 容器内。

3.1. 查看显卡

```
#查看显卡  
nvidia-smi
```

3.2. 上传数据集到服务器上

需要事先将数据集打包压缩，通过 sttp 上传至服务器。

在源码同级目录下创建 datasets 文件夹，如下：

```
(yolov8) root@668620c2db5e:/yolov8# ls
datasets gitee ultralytics ultralytics.zip
(yolov8) root@668620c2db5e:/yolov8#
(yolov8) root@668620c2db5e:/yolov8#
```

拷贝数据集至 datasets 文件夹内解压，其中[test_file]为数据集压缩文件名。

```
cd datasets
unzip [test_file]
```

3.3. 修改 yaml 文件

执行以下命令拷贝 coco128.yaml 文件到源码目录下，修改文件名并修改内容，注意源码 ultralytics 文件夹下还有个 ultralytics 文件夹。

```
cp ./ultralytics/datasets/coco128.yaml ./
mv coco128.yaml test.yaml
vim test.yaml
```

未修改的 test.yaml 文件。

```
1 # Ultralytics YOLO v8, GPL-3.0 license
2 # COCO128 dataset https://www.kaggle.com/ultralytics/coco128 (first 128 images from COCO train2017) by Ultralytics
3 # Example usage: yolo train data=coco128.yaml
4 # parent
5 #   └─ ultralytics
6 #       └─ datasets
7 #           └─ coco128 -- downloads here (7 MB)
8
9
10 # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ..]
11 path: ../datasets/coco128 # dataset root dir
12 train: images/train2017 # train images (relative to 'path') 128 images
13 val: images/train2017 # val images (relative to 'path') 128 images
14 test: # test images (optional)
15
16 # Classes
17 names:
18 0: person
19 1: bicycle
20 2: car
21 3: motorcycle
22 4: airplane
23 5: bus
24 6: train
25 7: truck
26 8: boat
27 9: traffic light
28 10: fire hydrant
29 11: stop sign
30 12: parking meter
31 13: bench
32 14: bird
33 15: cat
34 16: dog
35 17: horse
36 18: sheep
37 19: cow
38 20: elephant
39 21: bear
40 22: zebra
41 23: giraffe
42 24: backpack
43 25: umbrella
44 26: handbag
45 27: tie
46 28: suitcase
47 29: frisbee
48 30: skis
49 31: snowboard
50 32: sports ball
51 33: kite
52 34: baseball bat
53 35: baseball glove
54 36: skateboard
55 37: surfboard
56 38: tennis racket
57 39: bottle
58 40: wine glass
```

path:数据集根目录地址，运行时原地址是在源码路径下，使用相对路径或绝对路径均可

数据集根目录地址下的训练集图片地址

数据集根目录下的验证集图片地址

数据集根目录下测试集图片地址

类名，根据自己的数据集序号与类名相对应即可

修改后的 test.yaml 文件。

```
# Ultralytics YOLO 🚀, GPL-3.0 license
# COCO128 dataset https://www.kaggle.com/ultralytics/coco128 (first 128 i
# Example usage: yolo train data=coco128.yaml
# parent
# └─ ultralytics
# └─ datasets
# └─ coco128 ← downloads here (7 MB)

# Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt,
path: ../datasets/yolo # dataset root dir
train: images/train # train images (relative to 'path') 128 images
val: images/val # val images (relative to 'path') 128 images
test: images/test # test images (optional)

# Classes
names:
  0: test
```

3.4. 训练

切换到 yolov8 源码目录下训练，训练结果在源码的 runs 目录下。

#这里添加 0,1,2,3 四个显卡训练，添加参数每次批次图像数量 batch=16，需要与显卡数量成整数倍
yolo detect mode=train data=./test.yaml model=yolov8n.pt epochs=150 batch=16
imgsz=640 device=0,1,2,3

参考：<https://docs.ultralytics.com/zh/models/yolov8/#supported-tasks-and-modes>

3.5. 预测

使用训练出来的模型预测测试集。

#model 为模型路径，source 为测试集路径，conf 为阈值，0.5 代表置信度高于 0.5 时标出识别框
yolo detect predict model=./runs/detect/train6/weights/best.pt
source=./datasets/HandGesture/4 save=True conf=0.5

结果在源码 runs 目录下。

3.6. onnx 模型导出

修改 test.py 文件的输入模型路径。添加以下语句导出 onnx 模型。

结果在模型输入路径的目录下。

```
success = model.export(format="onnx", opset=12)
```

4. onnx 模型转换

以下是将 onnx 模型转换成板端运行的 om 格式模型的方法。

4.1. 安装 atc 相关环境

```
conda create -n atc python=3.9.2
conda env list
conda config --set auto_activate_base false
conda activate atc
conda deactivate
```

4.2. 安装 atc 相关依赖

```
pip3 install protobuf==3.13.0 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install psutil==5.7.0 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install numpy --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install scipy --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install decorator==4.4.0 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install sympy==1.5.1 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install cffi==1.12.3 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install pyyaml --user -i https://pypi.tuna.tsinghua.edu.cn/simple
pip3 install pathlib2 --user -i https://pypi.tuna.tsinghua.edu.cn/simple
```

4.3. 安装 atc 软件

参考官方 SDK 文件夹下：NNN/驱动和开发环境安装指南。

先修改安装文件权限

```
chmod +X Ascend-cann-toolkit_5.20.t6.2.b060_linux-x86_64.run
```

安装 cann 软件

```
./Ascend-cann-toolkit_5.20.t6.2.b060_linux-x86_64.run --install
```

安装完成后运行以下命令：

```
source  
/usr/local/Ascend/ascend-toolkit/5.20.t6.2.b060/x86_64-linux/bin/setenv.bash
```

其中/usr/local/Ascend/ascend-toolkit/为安装目录。

4.4. YOLOV8 转换模型

YOLOV8 转换命令如下：

```
Source  
$HOME/Ascend/ascend-toolkit/latest/x86_64-linux/bin/setenv.bash  
atc --model=yolov5s_v6.2.onnx --framework=5 --output=yolov5s_v6.2 --soc_version="OPTG"  
--output_type=FP32 --insert_op_conf=./op.cfg
```

示例代码存在 doc/act/onnx2om.sh.可以直接运行以下命令进行转换。

./onnx2om.sh 输入模型文件名 输出模型文件名

4.5. op. cfg 文件

设置 yuv420 输入配置,输入为 640*640 yuv420

```
aipp_op {  
    aipp_mode : static  
    related_input_rank : 0  
    max_src_image_size : 1228800  
    support_rotation : false  
    input_format : YUV420SP_U8  
    src_image_size_w : 640  
    src_image_size_h : 640  
    cpadding_value : 0.0  
    crop : false  
    load_start_pos_w : 0  
    load_start_pos_h : 0  
    crop_size_w : 0  
    crop_size_h : 0  
    resize : false
```

```
resize_output_w : 640
resize_output_h : 640
padding : false
left_padding_size : 0
right_padding_size : 0
top_padding_size : 0
bottom_padding_size : 0
padding_value : 0
csc_switch : true
rbuv_swap_switch : false
ax_swap_switch : false
matrix_r0c0 : 256
matrix_r0c1 : 0
matrix_r0c2 : 0
matrix_r1c0 : 0
matrix_r1c1 : 0
matrix_r1c2 : 0
matrix_r2c0 : 0
matrix_r2c1 : 0
matrix_r2c2 : 0
output_bias_0 : 0
output_bias_1 : 0
output_bias_2 : 0
input_bias_0 : 0
input_bias_1 : 0
input_bias_2 : 0
mean_chn_0 : 0
min_chn_0 : 0.0
var_rec1_chn_0 : 0.00392157
```

5. 板端推理

将生成的 **om** 离线模型、拷贝至板端进行模型推理即可完成板端部署。

5.1. 运行方式

拷贝到板端并命名为 yolov8n.om, 或者修改代码。

注：代码中默认只会在置信度 ≥ 0.85 时绘制框，可将阈值调低至 0.50 进行测试。

```
root@localhost:/opt/sample/yolov8# ls
sample_yolov8_imx347  sample_yolov8_os04a10  sample_yolov8_os08a20  yolov8.om
root@localhost:/opt/sample/yolov8# mv yolov8.om yolov8n.om
root@localhost:/opt/sample/yolov8# ls
sample_yolov8_imx347  sample_yolov8_os04a10  sample_yolov8_os08a20  yolov8n.om
root@localhost:/opt/sample/yolov8#
```

```
export LD_LIBRARY_PATH=/opt/lib/npu:$LD_LIBRARY_PATH
```

```
./sample_yolov8_imx347
```



易百纳公司名称：南京启诺信息技术有限公司

易百纳技术社区：www.ebaina.com

技术咨询电话：18013825199

技术服务微信：david089968

